Universal-ESP32-Workbench REST API

Scope and source

This documentation is based on the actual implementation in the repository, not just on README text. The primary sources analyzed were:

- `pi/portal.py` (central HTTP server and all route definitions)
- `pi/wifi_controller.py`
- `pi/ble_controller.py`
- `pi/debug_controller.py`
- `pi/cw_beacon.py`
- `CLAUDE.md`
- `.claude/skills/workbench-wifi/SKILL.md`
- `.claude/skills/workbench-ble/SKILL.md`
- `.claude/skills/workbench-logging/SKILL.md`
- `.claude/skills/workbench-test-handling/SKILL.md`

Base URL: `http://workbench.local:8080`

Key global observations

- The API is a custom HTTP server built on `http.server`, not a Flask/FastAPI application.
- JSON is the default format. Exceptions: `POST /api/firmware/upload` (`multipart/form-data`) and `GET /firmware/{project}/{filename}` (binary download).
- CORS is globally open: `Access-Control-Allow-Origin: *`; `OPTIONS` allows `GET, POST, DELETE, OPTIONS`.
- Many runtime failures are **not** returned as HTTP error status codes, but as HTTP 200 with `{"ok": false, "error": ...}`. This is how the code is actually implemented, and clients should account for it.
- `POST /api/serial/monitor` uses **substring matching**, not regex. The name `pattern` is misleading.
- `POST /api/enter-portal` works asynchronously: the call only starts the workflow and returns immediately. Progress is then available via `GET /api/log`.
- `POST /api/human-interaction` is intentionally blocking: the HTTP request remains open until confirmed, canceled, or timed out.
- In the current `portal.py`, there are **no** publicly exposed `/api/mqtt/*` routes, even though `mqtt_controller.py` and an MQTT skill structure exist in the repo.

States and shared data structures

Slot status (`/api/devices`)

Possible `state` values according to the code:

- `absent`
- `idle`
- `resetting`
- `monitoring`
- `flapping`
- `recovering`

- `download_mode`
- `debugging`

Typical slot fields in the response:

- `label`
- `slot_key`
- `tcp_port`
- `gdb_port`
- `openocd_telnet_port`
- `group` , `role`
- `gpio_boot` , `gpio_en` , `has_gpio`
- `present` , `running` , `pid` , `devnode` , `devnodes`
- `url` , `last_error` , `flapping` , `recovering` , `recover_retries` , `state`
- `detected_chip` , `jtag_slot`
- `debugging` , `debug_chip` , `debug_gdb_port`
- `usb_devices` , `usb_warning` , `is_probe`

Route inventory

Core

Method	Path	Purpose
GET	/api/devices	List all slots/devices and host metadata
GET	/api/info	Portal host info and slot counters
POST	/api/hotplug	Internal hotplug event ingress
POST	/api/start	Low-level start of RFC2217 proxy for a slot_key
POST	/api/stop	Low-level stop of RFC2217 proxy for a slot_key
GET	/api/log	Activity log

Serial

Method	Path	Purpose
POST	/api/serial/reset	Reset a slot and capture boot output
POST	/api/serial/monitor	Read serial output, optionally waiting for a substring
GET	/api/serial/output	Read the passive serial ring buffer
POST	/api/serial/recover	Manually trigger USB flap recovery
POST	/api/serial/release	Release BOOT GPIO after download mode flashing

WiFi

Method	Path	Purpose
GET	/api/wifi/ping	WiFi controller version and uptime
GET	/api/wifi/mode	Read WiFi operating mode
POST	/api/wifi/mode	Switch WiFi operating mode

Method	Path	Purpose
POST	/api/wifi/ap_start	Start workbench AP
POST	/api/wifi/ap_stop	Stop workbench AP
GET	/api/wifi/ap_status	Read AP state and connected stations
POST	/api/wifi/sta_join	Join external WiFi as station
POST	/api/wifi/sta_leave	Leave external WiFi; restore saved AP if any
GET	/api/wifi/scan	Scan nearby WiFi networks
GET	/api/wifi/events	Long-poll WiFi station connect/disconnect events
POST	/api/wifi/http	HTTP relay via the Pi
POST	/api/wifi/lease_event	Internal dnsmasq lease callback
POST	/api/enter-portal	Asynchronous captive-portal provisioning workflow

Human/Test

Method	Path	Purpose
POST	/api/human-interaction	Blocking operator prompt
GET	/api/human/status	Current operator prompt state
POST	/api/human/done	Complete pending operator prompt
POST	/api/human/cancel	Cancel pending operator prompt
GET	/api/test/progress	Current UI test session state
POST	/api/test/update	Start/update/end test session

GPIO

Method	Path	Purpose
GET	/api/gpio/status	Current requested GPIO pin state
POST	/api/gpio/set	Drive/release a GPIO pin

Debug

Method	Path	Purpose
GET	/api/debug/status	Debug/OpenOCD session status per slot
GET	/api/debug/probes	Configured external debug probes
GET	/api/debug/group	Slot groups and roles from config
POST	/api/debug/start	Start OpenOCD/GDB session
POST	/api/debug/stop	Stop OpenOCD/GDB session

CW

Method	Path	Purpose
GET	/api/cw/status	Current CW beacon state
GET	/api/cw/frequencies	List achievable integer-divider frequencies
POST	/api/cw/start	Start CW beacon

Method	Path	Purpose
POST	/api/cw/stop	Stop CW beacon

UDP Log

Method	Path	Purpose
GET	/api/udplog	Read buffered UDP log lines
DELETE	/api/udplog	Clear UDP log buffer

Firmware

Method	Path	Purpose
GET	/api/firmware/list	List uploaded firmware files
GET	/firmware/{project}/{filename}	Download firmware binary
POST	/api/firmware/upload	Upload firmware binary
DELETE	/api/firmware/delete	Delete firmware binary (JSON body)

BLE

Method	Path	Purpose
GET	/api/ble/status	BLE availability/connection state
POST	/api/ble/scan	Scan for BLE peripherals
POST	/api/ble/connect	Connect to BLE peripheral
POST	/api/ble/disconnect	Disconnect current BLE peripheral
POST	/api/ble/write	Write bytes to a GATT characteristic

Detailed documentation

Core / Portal

`GET /api/devices`

Returns the current state of all slots plus host metadata.

Request No parameters.

Success HTTP 200. Example structure:

```
{
  "slots": [{
    "label": "SLOT1",
    "slot_key": "platform-...",
    "state": "idle",
    "running": true,
    "url": "rfc2217://192.168.0.10:4001",
    "detected_chip": "esp32s3",
    "debugging": true,
    "debug_chip": "esp32s3",
    "debug_gdb_port": 3333,
    "devnodes": ["/dev/ttyACM0", "/dev/ttyACM1"],
    "usb_devices": [{"product": "USB JTAG/serial debug unit", "vid_pid": "303a:1001"}]
  }],
  "host_ip": "192.168.0.10",
  "hostname": "workbench"
}
```

Errors / status codes - HTTP 200 in the implementation. - Unknown paths globally end with HTTP 404 `{"error": "not found"}`.

Example

```
curl http://workbench.local:8080/api/devices
```

`GET /api/info`

Compact health/info endpoint for discovery and scripts.

Request No parameters.

Success HTTP 200:

```
{
  "host_ip": "192.168.0.10",
  "hostname": "workbench",
  "slots_configured": 3,
  "slots_running": 1
}
```

Errors / status codes No explicit error paths in the handler.

Example

```
curl http://workbench.local:8080/api/info
```

`POST /api/hotplug`

Internal endpoint for udev/hotplug events. It maps devices to slots, starts/stops proxies, and triggers recovery.

Request JSON body:

- `action` *(required)*: `add` or `remove`

- `devnode` *(optional, typical)*: e.g. `/dev/ttyACM0`
- `id_path` *(optional)*: preferred slot key
- `devpath` *(optional)*: fallback slot key

At least `id_path` or `devpath` must be set.

Success HTTP 200, z. B.:

```
{
  "ok": true,
  "slot_key": "platform-...",
  "seq": 42,
  "accepted": true,
  "flapping": false,
  "recovering": false
}
```

Errors / status codes - `400` for an empty body, missing `action`, or missing `id_path` / `devpath`. - `200` with `accepted: false` if the event is ignored (e.g. recovery is running or an add event arrives during debugging).

Notes For normal clients, this endpoint is generally **not** intended for interactive use.

Example

```
curl -X POST http://workbench.local:8080/api/hotplug \
  -H 'Content-Type: application/json' \
  -d '{"action":"add","devnode":"/dev/ttyACM0","id_path":"platform-3f980000.usb-usb-0:1.1.2:1.
```

`POST /api/start`

Low-level admin endpoint: starts the RFC2217 proxy for an existing `slot_key` and sets `devnode` / `present`.

Request JSON body:

- `slot_key` *(required)*
- `devnode` *(required)*

Success HTTP 200:

```
{
  "ok": true,
  "slot_key": "platform-...",
  "running": true
}
```

Errors / status codes - `400` for an empty body or missing `slot_key` / `devnode`. - `404` for an unknown `slot_key`. - `200` with `ok: false` is possible if the proxy fails to start technically.

Example

```
curl -X POST http://workbench.local:8080/api/start \
  -H 'Content-Type: application/json' \
  -d '{"slot_key":"platform-3f980000.usb-usb-0:1.1.2:1.0","devnode":"/dev/ttyACM0"}'
```

`POST /api/stop`

Low-level admin endpoint: stops the RFC2217 proxy for an existing `slot_key`.

Request JSON body:

- `slot_key` *(required)*

Success HTTP 200:

```
{
  "ok": true,
  "slot_key": "platform-...",
  "running": false
}
```

Errors / status codes - `400` for an empty body or missing `slot_key`. - `404` for an unknown `slot_key`.

Example

```
curl -X POST http://workbench.local:8080/api/stop \
  -H 'Content-Type: application/json' \
  -d '{"slot_key":"platform-3f980000.usb-usb-0:1.1.2:1.0"}'
```

`GET /api/log`

Returns the activity log of the workbench portal (not the device logs).

Request Query parameters:

- `since` *(optional)*: ISO 8601 timestamp; only newer entries are returned

Success HTTP 200:

```
{
  "ok": true,
  "entries": [
    {"ts": "2026-03-31T10:20:30+00:00", "msg": "WiFi scanning...", "cat": "step"},
    {"ts": "2026-03-31T10:20:31+00:00", "msg": "WiFi scan found 4 networks", "cat": "ok"}
  ]
}
```

Errors / status codes No explicit handler errors.

Notes According to the code, `cat` is typically `info`, `ok`, `error`, or `step`.

Example

```
curl 'http://workbench.local:8080/api/log?since=2026-03-31T00:00:00Z'
```

Serial / RFC2217

`POST /api/serial/reset`

Resets a device. If a debug session is active for the slot, the code first tries a JTAG reset; otherwise it toggles DTR/RTS directly on the serial interface. The response contains the captured boot lines.

Request JSON body:

- `slot` *(required)*: Slot-Label wie `SLOT1`

Success HTTP 200, z. B.:

```
{
  "ok": true,
  "output": [
    "ESP-ROM:esp32c3-api1-20210207",
    "boot:0x3 (DOWNLOAD_BOOT(UART0/UART1/SDIO_REI_REO_V2))"
  ]
}
```

Errors / status codes - `400` if `slot` is missing. - `200` with `ok: false` if the slot is unknown or the reset fails technically, e.g. `slot 'SLOT9' not found`, `SLOT1: no device node`, `Cannot open /dev/ttyACM0: ...`. - On JTAG reset failures, the code automatically falls back to DTR/RTS.

Example

```
curl -X POST http://workbench.local:8080/api/serial/reset \
  -H 'Content-Type: application/json' \
  -d '{"slot":"SLOT1"}'
```

`POST /api/serial/monitor`

Opens the active RFC2217 connection locally and reads serial output. Optionally waits, up to a timeout, for a line that contains `pattern` **as a substring**.

Request JSON body:

- `slot` *(required)*
- `pattern` *(optional)*: simple substring, **not regex**
- `timeout` *(optional, default `10`)*: number of seconds

Success HTTP 200:

```
{
  "ok": true,
  "matched": true,
  "line": "WiFi connected to TestAP",
  "output": ["booting...", "WiFi connected to TestAP"]
}
```

Errors / status codes - `400` if `slot` is missing. - `200` with `ok: false` if the slot is unknown, no `tcp_port` is configured, or the proxy is not running. - A timeout without a match is **not** an HTTP error: `ok: true`, `matched: false`.

Notes Actual matching in the code: `if pattern and pattern in stripped:`.

Example

```
curl -X POST http://workbench.local:8080/api/serial/monitor \
  -H 'Content-Type: application/json' \
  -d '{"slot":"SLOT1","pattern":"WiFi connected","timeout":15}'
```

`GET /api/serial/output`

Reads the passive ring buffer of a slot. This buffer is populated, among other times, after `serial/reset`.

Request Query parameters:

- `slot` *(required)*
- `lines` *(optional, default `50`)*
- `since` *(optional, default `0`)*: Unix-Timestamp (`float`)

Success HTTP 200:

```
{
  "ok": true,
  "lines": [
    {"ts": 1711879200.12, "text": "boot:0x3"},
    {"ts": 1711879200.12, "text": "waiting for download"}
  ]
}
```

Errors / status codes - `400` if `slot` is missing. - `200` with `ok: false` if the slot is unknown.

Example

```
curl 'http://workbench.local:8080/api/serial/output?slot=SLOT1&lines=50&since=0'
```

`POST /api/serial/recover`

Manually starts USB flap recovery for a slot.

Request JSON body:

- `slot` *(required)*

Success HTTP 200:

```
{
  "ok": true,
  "message": "recovery started for SLOT1"
}
```

Errors / status codes - `400` if `slot` is missing. - `200` with `ok: false` if the slot is unknown.

Example

```
curl -X POST http://workbench.local:8080/api/serial/recover \
  -H 'Content-Type: application/json' \
  -d '{"slot":"SLOT1"}'
```

`POST /api/serial/release`

Releases BOOT/GPIO after a download-mode recovery and optionally pulses EN for a clean restart.

Request JSON body:

- `slot` *(required)*

Success HTTP 200 on success:

```
{
  "ok": true
}
```

Errors / status codes - `400` if `slot` is missing. - `200` with `ok: false` if the slot is unknown, no `gpio_boot` is configured, or the slot is not in state `download_mode`.

Example

```
curl -X POST http://workbench.local:8080/api/serial/release \
  -H 'Content-Type: application/json' \
  -d '{"slot":"SLOT1"}'
```

WiFi

`GET /api/wifi/ping`

Fast status endpoint of the WiFi controller.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "fw_version": "1.0.0-pi",
  "uptime": 1234
}
```

Errors / status codes No explicit handler errors.

Example

```
curl http://workbench.local:8080/api/wifi/ping
```

`GET /api/wifi/mode`

Reads the current WiFi operating mode of the Pi.

Request No parameters.

Success HTTP 200:

- `wifi-testing` : `{"ok": true, "mode": "wifi-testing"}`
- `serial-interface` : `{"ok": true, "mode": "serial-interface", "ssid": "MyNetwork", "ip": "192.168.0.20"}`

Errors / status codes No explicit handler errors.

Example

```
curl http://workbench.local:8080/api/wifi/mode
```

`POST /api/wifi/mode`

Switches between `wifi-testing` and `serial-interface`. In `serial-interface` mode, the actual WiFi test endpoints are effectively disabled.

Request JSON body:

- `mode` *(required)*: `wifi-testing` or `serial-interface`
- `ssid` *(required for `serial-interface`)*
- `pass` *(optional)*

Success HTTP 200, z. B.:

```
{
  "ok": true,
  "mode": "serial-interface",
  "ssid": "OfficeWiFi",
  "ip": "192.168.0.20"
}
```

Errors / status codes - `400` for an empty body or invalid `mode`. - `200` with `ok: false` for runtime errors, e.g. missing `ssid` for `serial-interface` or join failures.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/mode \
  -H 'Content-Type: application/json' \
  -d '{"mode":"serial-interface","ssid":"OfficeWiFi","pass":"secret"}'
```

`POST /api/wifi/ap_start`

Starts a SoftAP on `wlan0`, including `hostapd` and `dnsmasq`.

Request JSON body:

- `ssid` *(required)*

- `pass` *(optional)*
- `channel` *(optional, default* `6` *)*

Success HTTP 200:

```
{
  "ok": true,
  "ip": "192.168.4.1"
}
```

Errors / status codes - `400` for an empty body or missing `ssid`. - `200` with `ok: false` for runtime errors, e.g. `hostapd failed to start: ...`, `dnsmasq failed to start: ...`, or `WiFi testing disabled (Serial Interface mode)`.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/ap_start \
  -H 'Content-Type: application/json' \
  -d '{"ssid":"TestAP","pass":"testpass123","channel":6}'
```

`POST /api/wifi/ap_stop`

Stops the SoftAP.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes - `200` with `ok: false` for runtime errors. - No dedicated 4xx validation error exists.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/ap_stop
```

`GET /api/wifi/ap_status`

Reads AP state and the stations registered via DHCP lease events.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "active": true,
  "ssid": "TestAP",
  "channel": 6,
  "stations": [{"mac": "aa:bb:cc:dd:ee:ff", "ip": "192.168.4.2"}]
}
```

Errors / status codes No explicit handler errors.

Example

```
curl http://workbench.local:8080/api/wifi/ap_status
```

`POST /api/wifi/sta_join`

Connects `wlan0` as a station to an existing Wi-Fi network. If an AP was active before, the controller remembers that AP configuration for `sta_leave`.

Request JSON body:

- `ssid` *(required)*
- `pass` *(optional)*
- `timeout` *(optional, default `15`)*

Success HTTP 200:

```
{
  "ok": true,
  "ip": "192.168.0.20",
  "gateway": "192.168.0.1"
}
```

Errors / status codes - `400` for an empty body or missing `ssid`. - `200` with `ok: false` for runtime errors, e.g. `Failed to connect to 'SSID' within 15s`, `Connected to 'SSID' but no IP obtained`, or disabled WiFi testing mode.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/sta_join \
  -H 'Content-Type: application/json' \
  -d '{"ssid":"OfficeWiFi","pass":"secret","timeout":15}'
```

`POST /api/wifi/sta_leave`

Disconnects the station connection. If an AP was active before `sta_join`, it is restored automatically.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes - `200` with `ok: false` for runtime errors. - No dedicated 4xx validation error exists.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/sta_leave
```

`GET /api/wifi/scan`

Scans networks via `iw dev wlan0 scan -u` and returns SSID/RSSI/auth information, sorted by descending signal strength.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "networks": [
    {"ssid": "TestAP", "rssi": -42, "auth": "WPA2"},
    {"ssid": "Guest", "rssi": -67, "auth": "OPEN"}
  ]
}
```

Errors / status codes - `200` with `ok: false` for disabled WiFi testing mode or other runtime errors. - On scan timeout inside the controller: an empty network list is returned instead of an error.

Example

```
curl http://workbench.local:8080/api/wifi/scan
```

`GET /api/wifi/events`

Returns DHCP/station events from the internal queue. Optional long polling is supported.

Request Query parameters:

- `timeout` *(optional)*: seconds as `float`; if > 0 and the queue is empty, the first read blocks for that duration

Success HTTP 200:

```
{
  "ok": true,
  "events": [
    {"type": "STA_CONNECT", "mac": "aa:bb:cc:dd:ee:ff", "ip": "192.168.4.2", "hostname": "dut"

    {"type": "STA_DISCONNECT", "mac": "aa:bb:cc:dd:ee:ff"}
  ]
}
```

Errors / status codes No explicit handler errors. Invalid `timeout` values are silently ignored and treated as `0`.

Example

```
curl 'http://workbench.local:8080/api/wifi/events?timeout=30'
```

`POST /api/wifi/http`

Executes an HTTP request **from the Pi itself**. This allows devices in the isolated test network to be reached. Response bodies are returned Base64-encoded.

Request JSON body:

- `method` *(optional, default `GET`)*

- `url` *(required)*
- `headers` *(optional, object)*
- `body` *(optional)*: Base64-encoded request body
- `timeout` *(optional, default `10`)*

Success HTTP 200:

```
{
  "ok": true,
  "status": 200,
  "headers": {"Content-Type": "application/json"},
  "body": "eyJzdGF0dXMiOiAib2sifQ=="
}
```

Errors / status codes - `400` for an empty body or missing `url`. - **Important:** target HTTP errors (e.g. 404 from the DUT) are **not** treated as API errors; the API still returns `ok: true` and only sets `status` accordingly. - `200` with `ok: false` only for transport/URL/runtime errors, e.g. `HTTP request failed: ...` or disabled WiFi testing mode.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/http \
  -H 'Content-Type: application/json' \
  -d '{"method":"GET","url":"http://192.168.4.2/status","timeout":10}'
```

`POST /api/wifi/lease_event`

Internal callback endpoint for `dnsmasq` lease scripts. Updates the station list and event queue.

Request JSON body:

- `action` *(required)*: typischerweise `add`, `old`, `del`
- `mac` *(required)*
- `ip` *(optional)*
- `hostname` *(optional)*

Success HTTP 200: `{"ok": true}`

Errors / status codes - `400` for an empty body or missing `action` / `mac`.

Notes Usually not relevant for normal API clients; typically used by `wifi-lease-notify.sh`.

Example

```
curl -X POST http://workbench.local:8080/api/wifi/lease_event \
  -H 'Content-Type: application/json' \
  -d '{"action":"add","mac":"aa:bb:cc:dd:ee:ff","ip":"192.168.4.2","hostname":"dut"}'
```

`POST /api/enter-portal`

Starts the asynchronous captive-portal provisioning workflow. The endpoint itself does **not** block until completion.

Request JSON body:

- `portal_ssid` *(optional, default `iOS-Keyboard-Setup`)*
- `portal_ip` *(optional, default `192.168.4.1`)*
- `ssid` *(required)*: target SSID entered into the portal
- `password` *(optional)*: target password entered into the portal

Success HTTP 200 sofort nach Start:

```
{
  "ok": true,
  "message": "enter-portal started in background"
}
```

Errors / status codes - `200` with `ok: false` for missing `ssid` or if another enter-portal run is already active. - Progress/errors are then written to the activity log, not returned in the synchronous response.

Notes The `workbench-wifi` skill describes the flow correctly; the actual implementation starts a background thread for it.

Example

```
curl -X POST http://workbench.local:8080/api/enter-portal \
  -H 'Content-Type: application/json' \
  -d '{"portal_ssid":"iOS-Keyboard-Setup","ssid":"TestAP","password":"testpass123"}'
```

Human interaction / test tracking

`POST /api/human-interaction`

Blocking operator dialog. The HTTP connection remains open until `/api/human/done`, `/api/human/cancel`, or a timeout occurs.

Request JSON body:

- `message` *(required)*
- `timeout` *(optional, default `120`)*

Success HTTP 200 on completion:

- Confirmed: `{"ok": true, "confirmed": true}`
- Canceled: `{"ok": true, "confirmed": false}`
- Timeout: `{"ok": true, "confirmed": false, "timeout": true}`

Errors / status codes - `400` for missing `message`. - `409` if another human-interaction request is already pending.

Example

```
curl -X POST http://workbench.local:8080/api/human-interaction \
  -H 'Content-Type: application/json' \
  -d '{"message":"Press reset and click Done","timeout":60}'
```

`GET /api/human/status`

Zeigt an, ob aktuell eine Operator-Aktion aussteht.

Request No parameters.

Success HTTP 200:

- Pending: `{"ok": true, "pending": true, "message": "..."}`
- Idle: `{"ok": true, "pending": false, "message": ""}`

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/human/status
```

`POST /api/human/done`

Confirms the currently blocking human-interaction request.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes - `200` with `ok: false` and `error: "no pending request"` if nothing is pending.

Example

```
curl -X POST http://workbench.local:8080/api/human/done
```

`POST /api/human/cancel`

Cancels the currently blocking human-interaction request.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes - `200` with `ok: false` and `error: "no pending request"` if nothing is pending.

Example

```
curl -X POST http://workbench.local:8080/api/human/cancel
```

`GET /api/test/progress`

Reads the current test-session state for the UI.

Request No parameters.

Success HTTP 200:

- No session: `{"ok": true, "active": false}`
- Aktive Session, z. B.:

```
{
  "ok": true,
  "active": true,
  "spec": "iOS-Keyboard v1.0",
  "phase": "Phase 1",
  "total": 8,
  "completed": [{"id": "TC-001", "result": "PASS"}],
  "current": {"id": "TC-002", "name": "WiFi Provisioning"}
}
```

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/test/progress
```

`POST /api/test/update`

Starts, updates, or ends a test session. The endpoint accepts several different request body shapes.

Request JSON body variants:

- Start: `spec`, optional `phase`, `total`
- Phase-Update: `phase`
- Total-Update: `total`
- Aktueller Test: `current` (beliebiges Objekt)
- Result: `result` (arbitrary object; appended to `completed`)
- Ende: `end: true`

Success HTTP 200: `{"ok": true}`

Errors / status codes - `400` for an empty body. - `400` with `error: "no active session"` if an update is sent without a prior session start (`spec`). - `end: true` ends the session immediately.

Example

```
curl -X POST http://workbench.local:8080/api/test/update \
  -H 'Content-Type: application/json' \
  -d '{"spec":"iOS-Keyboard v1.0","phase":"Phase 1","total":8}'
```

GPIO

`GET /api/gpio/status`

Reads the state of all GPIO lines currently requested by the portal.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "pins": {
    "17": {"direction": "output", "value": 1},
    "18": {"direction": "input", "value": 1}
  }
}
```

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/gpio/status
```

`POST /api/gpio/set`

Sets a GPIO pin low/high or releases it as an input with pull-up (`"z"`).

Request JSON body:

- `pin` *(required, int)*
- `value` *(required)*: `0` , `1` , or `"z"`

Allowed BCM pins according to the code: `5, 6, 12, 13, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27` .

Success HTTP 200:

```
{
  "ok": true,
  "pin": 18,
  "value": 0
}
```

Errors / status codes - `400` for an empty body, missing `pin` / `value` , a disallowed pin, or invalid `value` . - `200` with `ok: false` for runtime errors from `gpiod` .

Example

```
curl -X POST http://workbench.local:8080/api/gpio/set \
  -H 'Content-Type: application/json' \
  -d '{"pin":18,"value":0}'
```

Debug / OpenOCD / GDB

`GET /api/debug/status`

Returns, per slot, whether a debug session is currently active. Active sessions include additional OpenOCD data.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "slots": {
    "SLOT1": {"debugging": true, "chip": "esp32c3", "gdb_port": 3333, "telnet_port": 4444, "pi

    "SLOT2": {"debugging": false}
  }
}
```

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/debug/status
```

`GET /api/debug/probes`

Returns the configured external debug probes from the slot/probe configuration.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "probes": [
    {"label": "PROBE1", "type": "esp-prog", "in_use": false, "slot": null}
  ]
}
```

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/debug/probes
```

`GET /api/debug/group`

Groups slots based on the configuration fields `group` and `role`.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "groups": {
    "ESP32S3-1": {
      "serial": {"label": "SLOT1", "tcp_port": 4001, "gdb_port": 3333, "present": true, "runni

      "debug":  {"label": "SLOT1-JTAG", "tcp_port": 4002, "gdb_port": 3334, "present": true, "

    }
  }
}
```

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/debug/group
```

`POST /api/debug/start`

Starts an OpenOCD/GDB debug session. Without `slot`, the first present slot is selected. Without `chip`, the code tries auto-detection. Without `probe`, it first tries USB-JTAG and then, if needed, a free external probe.

Request JSON body:

- `slot` *(optional)*
- `chip` *(optional, z. B.* `esp32c3`, `esp32s3`*)*
- `probe` *(optional, Label eines konfigurierten Probes)*

Success HTTP 200 on success:

```
{
  "ok": true,
  "slot": "SLOT1",
  "chip": "esp32c3",
  "gdb_port": 3333,
  "telnet_port": 4444,
  "probe": null,
  "gdb_target": "target extended-remote workbench.local:3333"
}
```

Errors / status codes - `404` if no device was found or the specified slot does not exist. - `200` with `ok: false` for auto-detect/OpenOCD/probe errors, e.g. `could not auto-detect chip type ...`, `probe 'PROBE1' already in use ...`, `openocd failed to start: ...`.

Example

```
curl -X POST http://workbench.local:8080/api/debug/start \
  -H 'Content-Type: application/json' \
  -d '{"slot":"SLOT1","chip":"esp32c3"}'
```

`POST /api/debug/stop`

Stops a debug session. Without `slot`, the first active session is stopped; if none exists, the handler still returns success.

Request JSON body:

- `slot` *(optional)*

Success HTTP 200:

- With an active session: `{"ok": true, "slot": "SLOT1"}`
- Ohne aktive Session: `{"ok": true}`

Errors / status codes No 4xx validation errors exist in the handler. Errors from `debug_controller.stop()` are not encoded separately; stop is idempotent.

Example

```
curl -X POST http://workbench.local:8080/api/debug/stop \
  -H 'Content-Type: application/json' \
  -d '{"slot":"SLOT1"}'
```

CW beacon

`GET /api/cw/status`

Reads the current state of the Morse/CW beacon.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "active": false,
  "pin": null,
  "freq_hz": 0,
  "divider": 0,
  "message": "",
  "wpm": 15,
  "repeat": false
}
```

Errors / status codes No explicit error paths.

Example

```
curl http://workbench.local:8080/api/cw/status
```

`GET /api/cw/frequencies`

Lists all frequencies reachable with an integer divider within a given range.

Request Query parameters:

- `low` *(optional, default* `3500000` *)*
- `high` *(optional, default* `4000000` *)*

Success HTTP 200:

```
{
  "ok": true,
  "frequencies": [
    {"divider": 143, "freq_hz": 3496503.4965034965},
    {"divider": 142, "freq_hz": 3521126.7605633805}
  ]
}
```

Errors / status codes No explicit handler errors. Invalid non-numeric query values would, however, raise an exception; there is no dedicated guard in the handler.

Example

```
curl 'http://workbench.local:8080/api/cw/frequencies?low=3500000&high=3600000'
```

`POST /api/cw/start`

Starts the hardware GPCLK CW beacon on GPIO 5 or 6.

Request JSON body:

- `freq` *(required)*
- `pin` *(optional, default* `5` *)*
- `message` *(optional, default empty; practically required because an empty string is treated as an error by the beacon code)*
- `wpm` *(optional, default* `15` *)*
- `repeat` *(optional, default* `true` *)*

Success HTTP 200 on success:

```
{
  "ok": true,
  "pin": 5,
  "freq_hz": 3561253.5612535616,
  "divider": 140,
  "message": "CQ TEST",
  "wpm": 15,
  "repeat": true
}
```

Errors / status codes - `400` for an empty body or missing `freq` . - `200` with `ok: false` for content-level errors from the beacon code, e.g. `Pin 7 has no GPCLK — use 5 or 6` , `Message is empty` , `WPM must be 1–60` , `Frequency ... Hz out of GPCLK range` .

Example

```
curl -X POST http://workbench.local:8080/api/cw/start \
  -H 'Content-Type: application/json' \
  -d '{"pin":5,"freq":3560000,"message":"CQ TEST","wpm":15,"repeat":true}'
```

`POST /api/cw/stop`

Stops the CW beacon.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes No explicit error paths.

Example

```
curl -X POST http://workbench.local:8080/api/cw/stop
```

UDP logs

`GET /api/udplog`

Reads the buffered UDP log store (maximum 2000 total lines kept in memory).

Request Query parameters:

- `since` *(optional, default `0`)*: Unix-Timestamp (`float`)
- `source` *(optional)*: Quell-IP filtern
- `limit` *(optional, default `200`)*

Success HTTP 200:

```
{
  "ok": true,
  "lines": [
    {"ts": 1711879200.12, "source": "192.168.4.2", "line": "OTA progress: 45%"}
  ]
}
```

Errors / status codes No explicit handler errors; however, the handler converts query values directly to `float` / `int`, so severely invalid values may be problematic server-side.

Example

```
curl 'http://workbench.local:8080/api/udplog?source=192.168.4.2&since=0&limit=50'
```

`DELETE /api/udplog`

Leert den UDP-Logpuffer.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes No additional error paths in the handler.

Example

```
curl -X DELETE http://workbench.local:8080/api/udplog
```

Firmware repository

`GET /api/firmware/list`

Lists all uploaded firmware files in the firmware directory.

Request No parameters.

Success HTTP 200:

```
{
  "ok": true,
  "files": [
    {"project": "ios-keyboard", "filename": "ios-keyboard.bin", "size": 1572864, "modified": "

  ]
}
```

Errors / status codes No explicit error paths; an empty list is valid.

Example

```
curl http://workbench.local:8080/api/firmware/list
```

`GET /firmware/{project}/{filename}`

Downloads a firmware file as binary data.

Request Path parameters:

- `project`
- `filename`

Success HTTP 200 with `Content-Type: application/octet-stream`, `Content-Disposition: attachment; filename=...`, and binary content.

Errors / status codes - `400` for invalid path format or a path traversal attempt (`..` or additional `/`). - `404` if the file does not exist.

Example

```
curl -OJ http://workbench.local:8080/firmware/ios-keyboard/ios-keyboard.bin
```

`POST /api/firmware/upload`

Uploads a file to the firmware repository. The parser is implemented manually and expects the exact

field names `project` and `file`.

Request Request-Typ: `multipart/form-data`

Form fields:

- `project` *(required)*
- `file` *(required)*

Success HTTP 200:

```
{
  "ok": true,
  "project": "ios-keyboard",
  "filename": "ios-keyboard.bin",
  "size": 1572864
}
```

Errors / status codes - `400` for wrong content type, missing boundary, empty body, missing `project` / `file`, or a path traversal attempt. - No dedicated 409/500 path exists in the handler.

Example

```
curl -X POST http://workbench.local:8080/api/firmware/upload \
  -F project=ios-keyboard \
  -F file=@build/ios-keyboard.bin
```

`DELETE /api/firmware/delete`

Deletes a firmware file. Unusually, this `DELETE` endpoint expects a JSON body.

Request JSON body:

- `project` *(required)*
- `filename` *(required)*

Success HTTP 200: `{"ok": true}`

Errors / status codes - `400` for an empty body, missing `project` / `filename`, or a path traversal attempt. - `404` if the file was not found.

Example

```
curl -X DELETE http://workbench.local:8080/api/firmware/delete \
  -H 'Content-Type: application/json' \
  -d '{"project":"ios-keyboard","filename":"ios-keyboard.bin"}'
```

BLE

`GET /api/ble/status`

Reads BLE availability and connection state.

Request No parameters.

Success HTTP 200:

- Unavailable: `{"ok": true, "state": "unavailable", "error": "bleak not installed"}`
- Idle: `{"ok": true, "state": "idle"}`
- Connected: `{"ok": true, "state": "connected", "address": "AA:BB:...", "name": "AA:BB:..."}`

Errors / status codes No 501 here: even when `bleak` is missing, the handler returns HTTP 200 with `state: "unavailable"`.

Example

```
curl http://workbench.local:8080/api/ble/status
```

`POST /api/ble/scan`

Scans for BLE peripherals with `bleak`. Results are sorted by RSSI in descending order.

Request JSON body *(optional)*:

- `timeout` *(optional; `<=0` means use the default from `BLE_SCAN_TIMEOUT`, which is 5.0s by default)*
- `name_filter` *(optional; simple substring match against the advertised name)*

Success HTTP 200 on success:

```
{
  "ok": true,
  "devices": [
    {"address": "AA:BB:CC:DD:EE:FF", "name": "iOS-Keyboard", "rssi": -42}
  ]
}
```

Errors / status codes - `501` if BLE/`bleak` is not available. - `200` with `ok: false` for scan errors or if a scan is already running (`scan already in progress`).

Example

```
curl -X POST http://workbench.local:8080/api/ble/scan \
  -H 'Content-Type: application/json' \
  -d '{"timeout":5,"name_filter":"iOS-Keyboard"}'
```

`POST /api/ble/connect`

Connects to a BLE device by address and returns the service/characteristic structure.

Request JSON body:

- `address` *(required)*

Success HTTP 200 on success:

```
{
  "ok": true,
  "address": "AA:BB:CC:DD:EE:FF",
  "name": "AA:BB:CC:DD:EE:FF",
  "services": [
    {"uuid": "6e400001-b5a3-f393-e0a9-e50e24dcca9e", "characteristics": [{"uuid": "6e400002-b5

  ]
}
```

Errors / status codes - `501` if BLE is not available. - `400` if `address` is missing. - `409` for any failed connect according to the handler, e.g. `already connected to ...` or `connect failed: ...`.

Example

```
curl -X POST http://workbench.local:8080/api/ble/connect \
  -H 'Content-Type: application/json' \
  -d '{"address":"AA:BB:CC:DD:EE:FF"}'
```

`POST /api/ble/disconnect`

Disconnects the current BLE connection. It is idempotent.

Request No body required.

Success HTTP 200: `{"ok": true}`

Errors / status codes - `501` if BLE is not available. - Otherwise, there are no additional error paths in the handler.

Example

```
curl -X POST http://workbench.local:8080/api/ble/disconnect
```

`POST /api/ble/write`

Writes hex data to a GATT characteristic. The request expects hex text, which is decoded into bytes server-side.

Request JSON body:

- `characteristic` *(required)*: UUID
- `data` *(required)*: Hex-String, Leerzeichen erlaubt
- `response` *(optional, default `true`)*: writes with or without response

Success HTTP 200 on success:

```
{
  "ok": true,
  "bytes_written": 5
}
```

Errors / status codes - `501` if BLE is not available. - `400` for missing `characteristic`, missing `data`, or invalid hex data. - `500` for write errors from `ble_controller.write()`, e.g. `not connected` or `write failed: ...`.

Example

```
curl -X POST http://workbench.local:8080/api/ble/write \
  -H 'Content-Type: application/json' \
  -d '{"characteristic":"6e400002-b5a3-f393-e0a9-e50e24dcca9e","data":"48656c6c6f","response":
```

Endpoints that are not REST API endpoints in the strict sense

- `GET /` and `GET /index.html` serve the built-in web UI.
- `OPTIONS *` returns global CORS responses.
- The UDP discovery responder on port `5888/udp` and the UDP log listener on `5555/udp` are network services, but not HTTP/REST endpoints.

Differences between skills/README and the actual code

The skills in the repo are useful as usage guidance, but the implementation in `portal.py` is the authoritative source. Important real details from the code:

1. `serial/monitor` uses substring search, not regex.
2. Many handlers return HTTP 200 with `ok: false` on runtime failures.
3. `ble/status` returns **200**, not 501, when BLE support is unavailable.
4. `wifi/http` treats target-device HTTP errors as a normal success of the relay operation and returns the target status in `status`.
5. There are currently no publicly routed MQTT endpoints in `portal.py`.